

so if say i want to work on a mathematical model for my research, its more importatn i have a well organised tagging system rather than a subagent descriptions?

Yes — for a research vault, a **well-designed note/link/tag/metadata system matters more than elaborate subagent descriptions**.

Subagents are useful, but they are secondary. They help split labor. They do not compensate for a messy knowledge base.

For mathematical-model research, I'd prioritize in this order:

- 1. **Clear note types**
Examples: model , assumption , variable , equation , derivation , dataset , simulation , paper , result , open-question .
- 2. **Consistent tags and YAML**
Tags help the agent find relevant material quickly. YAML helps it filter and reason structurally.
- 3. **Explicit links between concepts**
Use [[Model name]] , [[Assumption - linear demand]] , [[Parameter - beta]] , [[Derivation - stability condition]] , etc. These links are more valuable than the visual graph itself because the agent can traverse them.
- 4. **A strong AGENTS.md / vault instruction file**
This tells Codex how to treat your research notes, equations, evidence, and edits.
- 5. **Subagent descriptions**
Useful later for big tasks, such as "one agent checks assumptions, one reviews derivations, one searches papers, one inspects simulations."

A good system beats subagents because Codex can only coordinate well if the vault gives it stable handles.

For example, this is more useful:

```
---
type: model
status: active
domain: evolutionary game theory
related:
  - "[[Replicator dynamics]]"
  - "[[Stability analysis]]"
assumptions:
  - "[[Assumption - bounded rationality]]"
parameters:
  - alpha
  - beta
  - gamma
---
```

than a long subagent prompt saying "be a mathematical modeling expert" while your notes are inconsistently named and unlinked.

A simple tag system could be:

```
#type/model
#type/assumption
#type/equation
#type/derivation
#type/result
#type/paper
#type/simulation
#status/raw
#status/active
#status/reviewed
#status/question
#method/stability-analysis
```

```
#method/agent-based-model
#method/optimization
#domain/your-field
```

For Codex/Claudian, I'd put something like this in AGENTS.md :

```
# Research vault instructions

This vault supports mathematical-model research.

Treat notes as research objects:
- Models
- Assumptions
- Variables
- Equations
- Derivations
- Simulations
- Results
- Literature notes
- Open questions

Use YAML frontmatter and tags as structured metadata.
Treat [[wikilinks]] as conceptual dependencies.

When analyzing a model:
1. Identify the model note.
2. List variables, parameters, assumptions, equations, and claimed results.
3. Check whether every equation has definitions for its symbols.
4. Check whether assumptions are explicit.
5. Identify missing derivation steps.
6. Identify possible contradictions with linked notes.
7. Suggest tests, simulations, or proofs.
8. Do not rewrite equations unless asked.
9. Preserve LaTeX formatting.
10. Cite source notes using [[wikilinks]].

Do not rename, delete, or move notes without asking.
For large edits, produce a report before modifying files.
```

Then use subagents only when the task is large enough:

```
Use separate passes to review this model:
1. Derivation checker
2. Assumption checker
3. Literature consistency checker
4. Simulation/design checker

Do not edit notes yet. Produce a structured review with links to relevant notes.
```

My recommendation: build a **minimal but consistent research schema first**, then add subagent workflows once the vault has enough structure. For math/modeling work, the killer feature is not “many agents”; it is giving one good agent a clean graph of models, assumptions, variables, derivations, and evidence.

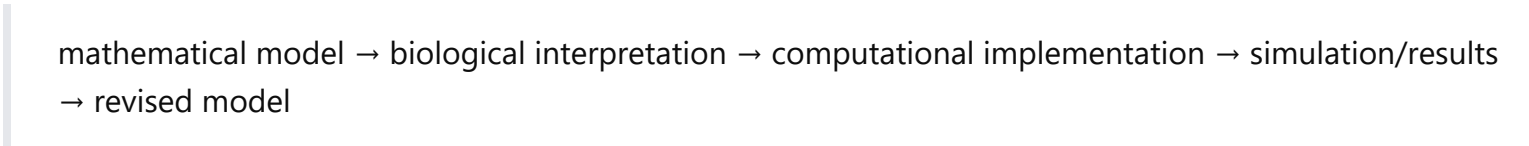
so if i have one vault where i do mathematical modeling, code implementation based on that model, biological domain implementation whats the best way to work together with codex or claude code agents/codex md and what matters at the vault level, ie seperate folders for the roles or rather seperate vaults per role but combine it in the codex/code md agentic system?

I'll frame this as an architecture decision for the vault, because the big choice is not “folders versus agents” but where the source of truth lives and how much cross-domain linking you need.

Nachgedacht für 12s

For your case, I would **not split into separate vaults by role**. I'd use **one integrated research vault/repo**, with **separate top-level folders, schemas, and agent instructions per domain**.

Your work has strong cross-dependencies:



If you split those into separate vaults, you make it harder for Codex/Claude Code to follow the chain of reasoning. The agent will constantly lose context across boundaries.

Best architecture

Use one vault that is also a Git repo:

```
ResearchVault/  
  AGENTS.md  
  CLAUDE.md  
  README.md  
  
  00_Inbox/  
  10_Theory/  
    Models/  
    Assumptions/  
    Equations/  
    Derivations/  
    OpenQuestions/  
  
  20_Biology/  
    Mechanisms/  
    Phenotypes/  
    Pathways/  
    ExperimentalContext/  
    Literature/  
  
  30_Code/  
    src/  
    tests/  
    notebooks/  
    scripts/  
    pyproject.toml  
  
  40_Simulations/  
    configs/  
    outputs/  
    figures/  
    reports/  
  
  50_Results/  
    ModelChecks/  
    BiologicalInterpretations/  
    NegativeResults/  
    DraftFigures/  
  
  60_Maps/  
    MOC - Main Research Model.md  
    MOC - Biological Domain.md  
    MOC - Computational Implementation.md  
  
  90_Meta/  
    Glossary.md  
    Variable Registry.md  
    Parameter Registry.md  
    Modeling Conventions.md  
    Tag Taxonomy.md
```

This gives you **one shared context** while still giving each role a clean workspace.

Why one vault is better

The core artifact is not the code or the notes separately. The core artifact is the **model–implementation–biology chain**.

For example, you want Codex/Claude Code to understand:

```
[[Model - X]]
  depends on → [[Assumption - bounded growth]]
  formalized by → [[Equation - transition function]]
  implemented in → 30_Code/src/model_x.py
  tested by → 30_Code/tests/test_model_x.py
  simulated by → 40_Simulations/configs/model_x_baseline.yaml
  interpreted through → [[Biological Mechanism - Y]]
  produces → [[Result - baseline instability]]
```

That is exactly the kind of thing one vault handles well.

Separate vaults are better only when the domains are truly independent, or when one part must have different privacy/security rules.

Use both AGENTS.md and CLAUDE.md

For Codex, the project instruction file is AGENTS.md ; OpenAI says Codex reads it before work and uses it for durable project guidance.

For Claude Code, the main project memory file is CLAUDE.md . Anthropic’s docs say Claude Code reads CLAUDE.md , not AGENTS.md , but recommends making CLAUDE.md import AGENTS.md when a repo already uses it.

So do this:

```
# CLAUDE.md

@AGENTS.md

## Claude-specific notes

Use plan mode before large vault reorganizations.
Prefer report-first workflows before editing research notes.
```

That keeps Codex and Claude Code aligned without duplicating instructions.

Use folder-level instructions for roles

The best pattern is **one root instruction file + local role instructions**.

Root AGENTS.md defines universal rules:

```
# AGENTS.md

This is an integrated Obsidian research vault and code repository.

The project combines:
- mathematical modeling
- biological interpretation
- computational implementation
- simulations
- research writing

Treat Markdown notes, code, simulation configs, and result files as one connected research system.

Universal rules:
- Preserve YAML frontmatter.
- Preserve LaTeX formatting.
- Treat [[wikilinks]] as conceptual dependencies.
- Treat tags as structured metadata.
- Do not rename, delete, or move notes without asking.
- Do not rewrite scientific claims unless source notes support the change.
- For large edits, produce a report before modifying files.
- When editing code, run relevant tests if available.
- When editing model notes, check symbol definitions and assumptions.
- When interpreting biology, separate evidence from speculation.
```

Then put smaller AGENTS.md files inside important folders:

```
10_Theory/AGENTS.md
20_Biology/AGENTS.md
30_Code/AGENTS.md
40_Simulations/AGENTS.md
50_Results/AGENTS.md
```

This works especially well because Codex is designed around project and directory-specific instructions. OpenAI’s docs describe `AGENTS.md` as durable guidance for repository rules, build/test commands, review expectations, and directory-specific conventions.

For Claude Code, directory hierarchy matters too: Anthropic says Claude Code loads `CLAUDE.md` / `CLAUDE.local.md` files by walking up the directory tree from the working directory, and imported files can be referenced using `@` syntax.

Role folders matter more than subagents

Subagents are useful, but they should reflect a real structure that already exists in the vault.

Claude’s docs describe subagents as specialized assistants for focused subtasks, separate context, and parallel work. That is useful when you have a big review task, but it is not the foundation.

The foundation should be:

- 1. **stable folders**
- 2. **note types**
- 3. **YAML schema**
- 4. **link conventions**
- 5. **test/simulation commands**
- 6. **root and folder-level agent instructions**

Then subagents become powerful because each one knows what it is reviewing.

Recommended role structure

Use **roles as workflows**, not as separate vaults.

Role	Folder anchor	Agent behavior
Mathematical modeler	10_Theory/	Check assumptions, equations, derivations, symbol consistency
Biological domain analyst	20_Biology/	Check biological plausibility, mechanisms, evidence strength
Implementation engineer	30_Code/	Translate model into code, tests, reproducible scripts
Simulation analyst	40_Simulations/	Run/organize configs, outputs, figures, sensitivity analyses
Research synthesizer	50_Results/ , 60_Maps/	Connect model, code, biology, and findings into insight notes

You can later encode these as Claude Code subagents, but I would first encode them as **folder conventions and prompts**.

The key vault-level files

- 1. `README.md`

For humans and agents:

```
# ResearchVault

This vault contains an integrated mathematical, biological, and computational research project.

Primary research question:
...

Main model:
```

```
[[Model - Main]]

Current active implementation:
30_Code/src/...

Main MOCs:
- [[MOC - Main Research Model]]
- [[MOC - Biological Domain]]
- [[MOC - Computational Implementation]]
```

2. AGENTS.md

For Codex and shared agent behavior.

3. CLAUDE.md

For Claude Code, ideally importing AGENTS.md .

4. 90_Meta/Variable Registry.md

This is extremely important for mathematical modeling.

```
# Variable Registry

| Symbol | Meaning | Units | Domain | Defined in | Implemented as |
|---|---|---|---|---|---|
| x(t) | population state | cells / mL | nonnegative real | [[Equation - State Dynamics]] | `state.x` |
| beta | interaction coefficient | 1/time | positive real | [[Parameter - beta]] | `params.beta` |
```

5. 90_Meta/Parameter Registry.md

```
# Parameter Registry

| Parameter | Biological meaning | Units | Default | Range | Source | Code name |
|---|---|---|---|---|---|---|
| beta | interaction rate | 1/day | 0.2 | 0.01-1.0 | [[Paper - Smith 2024]] | `beta` |
```

6. 90_Meta/Model-Code Traceability.md

This is the bridge between Obsidian and coding agents:

```
# Model-Code Traceability

| Model object | Theory note | Code file | Test file | Simulation config |
|---|---|---|---|---|
| Main transition equation | [[Equation - Transition Function]] | `30_Code/src/model/transition.py` | `30_Code/
| Stability criterion | [[Derivation - Stability Criterion]] | `30_Code/src/analysis/stability.py` | `30_Code/t
```

This one file will massively improve agent performance.

Suggested folder-level AGENTS.md

10_Theory/AGENTS.md

```
# Theory instructions

This folder contains mathematical models, assumptions, equations, and derivations.

When working here:
- Preserve LaTeX.
- Check that every symbol is defined in [[Variable Registry]] or the note itself.
- Link assumptions explicitly.
- Distinguish theorem, conjecture, heuristic, and simulation-backed claim.
```


- Do not modify biological interpretation unless cross-checking with 20 Biology

20_Biology/AGENTS.md

Biology instructions

This folder contains biological mechanisms, evidence, literature notes, and domain interpretation.

When working here:

- Separate empirical evidence from hypothesis.
- Link claims to source notes.
- Mark uncertainty explicitly.
- Do not treat model assumptions as biological facts.
- When a biological mechanism affects model structure, link to the relevant model note.

30_Code/AGENTS.md

Code instructions

This folder contains implementation of the mathematical models.

When working here:

- Keep code traceable to model notes.
- Add comments linking important functions to theory notes where useful.
- Run relevant tests after changes.
- Do not change equations silently; update corresponding theory notes or report mismatch.
- Preserve reproducibility.

Should code live inside the Obsidian vault?

Usually yes, if the code is part of the same research project.

Obsidian can ignore or hide code folders if needed. Codex/Claude Code benefits from seeing the notes and code together. Git benefits too: commits can connect “changed equation,” “updated implementation,” and “new simulation result.”

The only reason I’d keep code outside the vault is if:

- the codebase is huge,
- it has many generated files,
- Obsidian indexing becomes slow,
- you need separate dependency/environment management,
- collaborators should access code but not notes.

A hybrid version is also good:

ResearchProject/
 vault/
 code/
 data/
 AGENTS.md
 CLAUDE.md

Then open the **project root** in Codex/Claude Code, and open only `vault/` in Obsidian. This is often the cleanest professional setup.

For you, I’d probably choose this:

ResearchProject/
 AGENTS.md
 CLAUDE.md
 README.md

 vault/

```
10_Theory/  
20_Biology/  
50_Results/  
60_Maps/  
90_Meta/  
  
code/  
  src/  
  tests/  
  notebooks/  
  
simulations/  
  configs/  
  outputs/  
  figures/  
  
data/  
  raw/  
  processed/
```

This avoids turning Obsidian into a full software IDE while still letting agents see everything.

My strongest recommendation

Use **one project root**, not separate vaults:

```
ResearchProject/  
  AGENTS.md  
  CLAUDE.md  
  vault/  
  code/  
  simulations/  
  data/
```

Open `vault/` in Obsidian. Open `ResearchProject/` in Codex or Claude Code.

That gives you the best of both worlds:

- Obsidian remains clean for PKM.
- Codex/Claude Code sees the full research system.
- Folder-level instructions can specialize behavior.
- Model notes can link to code files.
- Code can link back to model notes.
- Simulations/results remain traceable.

What matters most at vault/project level

In order:

1. **One integrated project root**
2. **Clear folder boundaries**
3. **Root** `AGENTS.md` **and** `CLAUDE.md`
4. **Model–code traceability file**
5. **Variable and parameter registries**
6. **Consistent note types and YAML**
7. **Stable naming conventions**
8. **Tests and reproducible simulation commands**
9. **MOCs for human navigation**
10. **Subagents**

Subagents come after structure. They are orchestration, not ontology.

The ideal mental model is:

Treat your research project like a repo, your vault like the conceptual layer, your code like the executable layer, and your `AGENTS.md` / `CLAUDE.md` files like the operating manual that keeps agents from confusing one layer for another.

